

ENGENHARIA DE SOFTWARE 2026

Introdução à Engenharia de *Software*

O *Software* como produto

Professor Anderson Luiz Barbosa



Tópicos da Aula

01

Visão Geral

Perguntas fundamentais sobre software

02

O Papel Evolutivo

A trajetória do software na indústria

03

Software: Definição e Características

Conceitos essenciais e propriedades únicas

04

Crise do Software

Os problemas que impulsionaram a engenharia

Visão Geral

Antes de mergulhar nos conceitos técnicos, precisamos responder às perguntas fundamentais que orientam toda a disciplina de Engenharia de Software.



O que é Software?

Muito além de código: instruções, dados e documentação.



Quem desenvolve?

Equipes multidisciplinares de engenheiros e analistas.



Por que é importante?

O software permeia praticamente todos os setores da sociedade.



Quais são os passos?

Processos sistemáticos garantem qualidade e previsibilidade.



Qual é o produto?

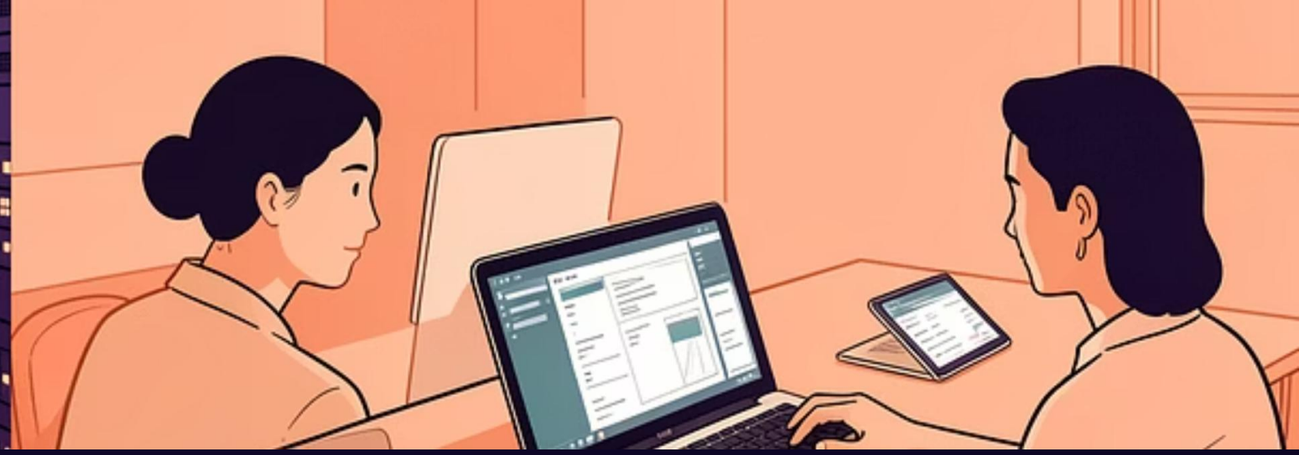
Programas, estruturas de dados e documentação completa.



Como validar a qualidade?

Testes, revisões e métricas de processo.





O Papel Evolutivo do *Software*

🖥️ Era do Hardware

No início da computação, o **hardware** representava o principal custo. A engenharia de hardware já contava com controles formais, métodos e ferramentas padronizados.

Enquanto isso, o *software* era visto como uma **"arte"** — o programador aprendia por tentativa e erro.

💻 Era do Software

Quando o software passou a ser o principal custo, surgiram questões críticas:

- Por que tanta demora para concluir os programas?
- Por que os custos são tão elevados?
- Por que erros são descobertos após a entrega?
- Por que é difícil medir o progresso?
- Por que gastamos tanto mantendo sistemas existentes?

Software — Definição

O software é composto por **três elementos fundamentais** que, juntos, formam um produto completo e funcional:



Instruções

Programas de computador que, quando executados, produzem a função e o desempenho desejados.



Estruturas de Dados

Possibilitam que os programas manipulem adequadamente as informações processadas.

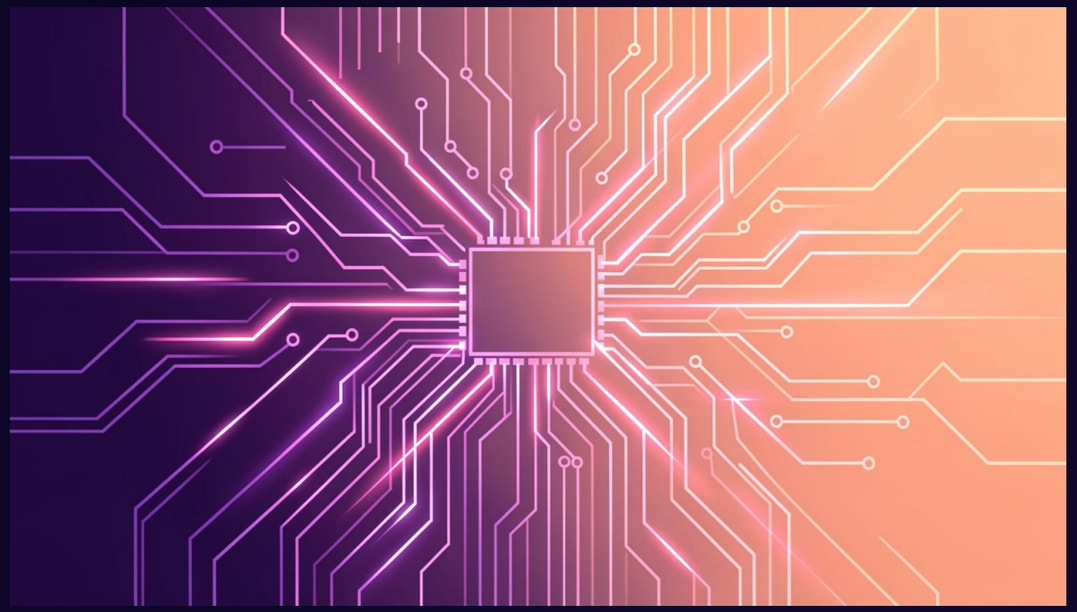


Documentação

Documentos que descrevem a operação, o uso e a manutenção dos programas.

SOFTWARE CARACTERÍSTICAS

Características do *Software*



O software possui propriedades únicas que o distinguem de produtos físicos:

- É um **elemento lógico**, não físico
- É **desenvolvido** por engenharia — não fabricado no sentido clássico
- Na maioria das vezes, é construído **sob encomenda**, apesar da ideia de componentes reutilizáveis
- É construído usando **linguagens de programação**

Evolução das Linguagens

1

1ª Geração
Linguagem de máquina

2

2ª Geração
Assembly

3

3ª Geração
Alto nível (Pascal, C...)

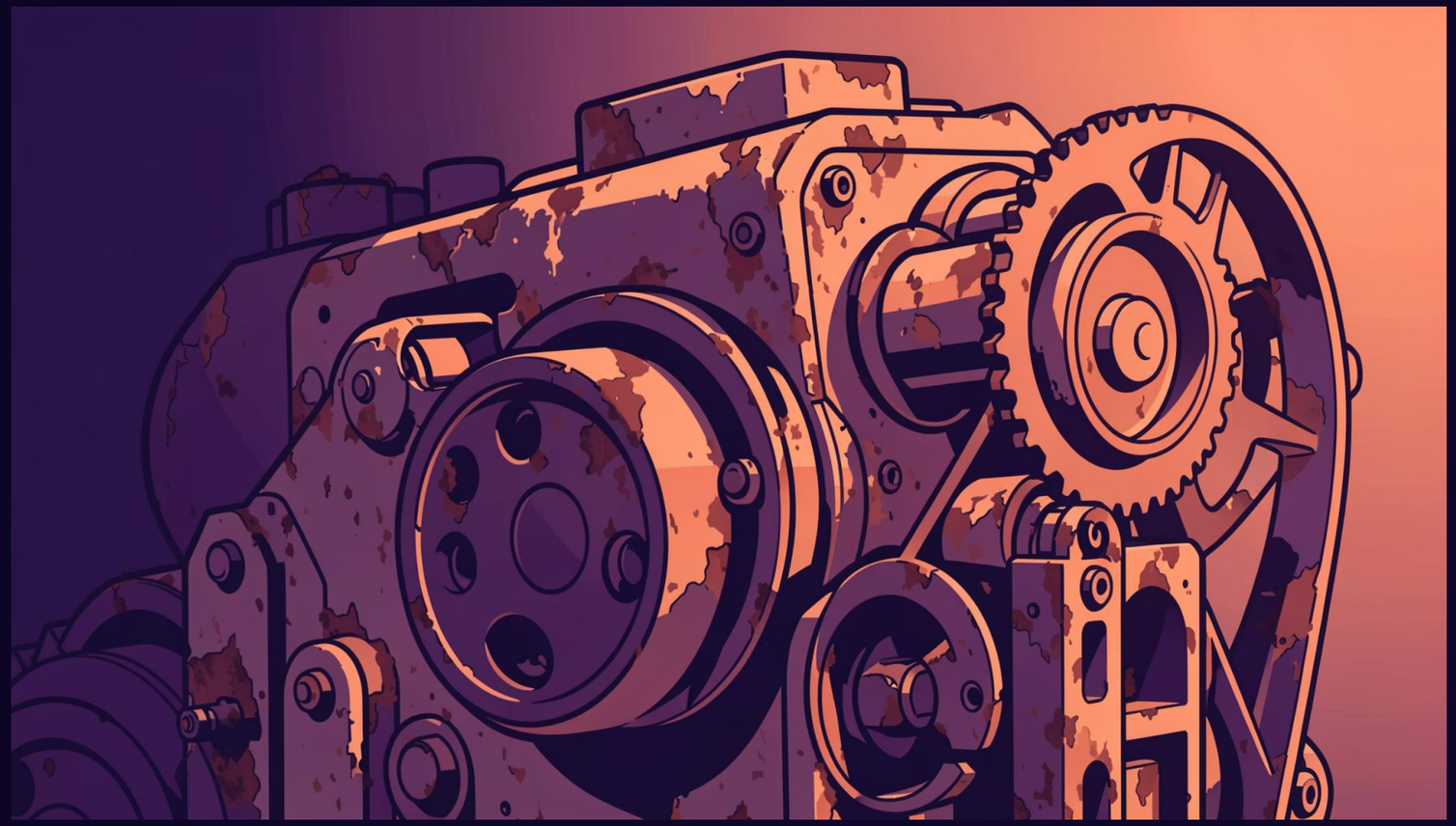
4

4ª Geração
Não procedimentais

SOFTWARE DESGASTE

Software Não "Se Desgasta"

Diferente do hardware, o software não sofre degradação física com o tempo. No entanto, isso não significa que ele esteja livre de problemas.



Hardware

Componentes físicos sofrem **desgaste natural**: corrosão, fadiga mecânica, deterioração. A taxa de falhas segue a famosa **"curva da banheira"**: alta no início (defeitos de fabricação), estável durante a vida útil, e crescente no fim.

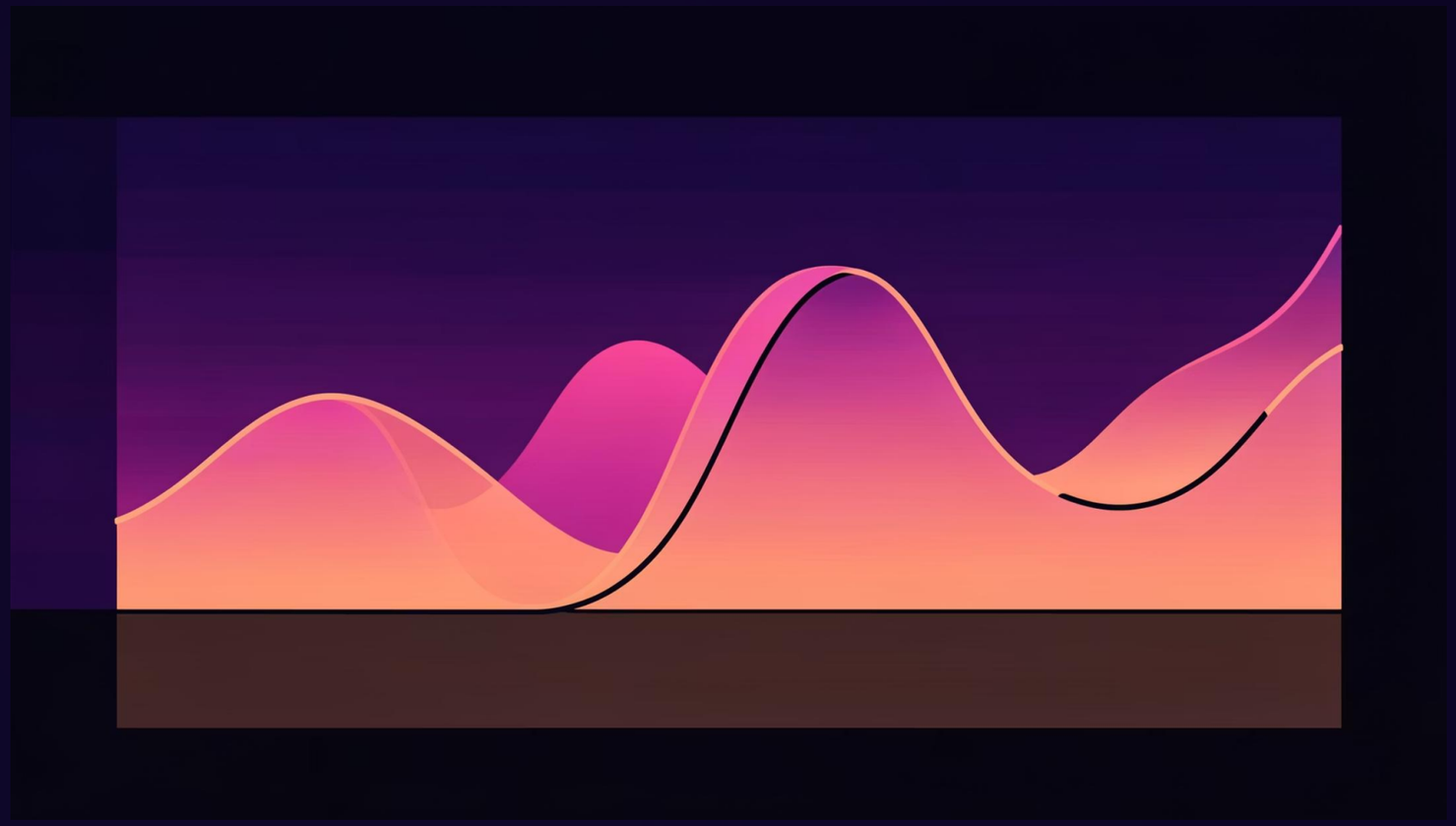
SOFTWARE COMPORTAMENTO IDEAL

A Curva Ideal do *Software*

Comportamento Teórico

Em teoria, após a correção de defeitos iniciais, a taxa de falhas do software deveria **estabilizar indefinidamente**, já que não há desgaste físico.

💡 O software não quebra como uma peça mecânica — ele falha por **defeitos de projeto**, não por uso.



SOFTWARE REALIDADE

A Realidade: Deterioração por Mudanças

Na prática, o software **deteriora-se com as modificações**. Cada alteração — correção, adaptação ou melhoria — pode introduzir novos defeitos, elevando progressivamente a taxa de falhas.



Versão Original **Mudanças Acumuladas** **Software Degradado**

Esse fenômeno é uma das razões pelas quais a **manutenção de software** consome grande parte dos recursos em projetos reais, exigindo práticas de engenharia disciplinadas.



A Crise do *Software*

A chamada "Crise do Software" revelou problemas sistêmicos que motivaram o surgimento da Engenharia de Software como disciplina formal.

⚠️ Questões Gerenciais

- Estimativas de prazos e custos **imprecisas**
- Produtividade não acompanha a demanda
- Qualidade de software inadequada

🛑 Problemas Recorrentes

- Faltam dados históricos sobre o desenvolvimento
- Frequente **insatisfação do cliente** com o produto final
- Qualidade sob suspeita constante
- Grande dificuldade na **manutenção** do software existente

Aplicações de *Software*

Software de Sistemas

Programas de apoio que interagem diretamente com o hardware (ex: compiladores, SO)

Software de Aplicação

Reestrutura dados para operações comerciais e tomada de decisão (ex: folha de pagamento, contas)



Aplicações de *Software* (continuação)



Software Científico e de Engenharia

Algoritmos de processamento numérico e projeto apoiado por computador. Aplicações em astronomia, genética e simulações complexas.



Software Embutido

Reside dentro do produto, controlando características específicas com restrições de memória e processamento. Ex: automóveis, micro-ondas.

Aplicações de *Software* (continuação)

Software de Linha de Produção

Capacidade específica para diferentes clientes:

- Mercado especial (ex: controle de estoque)
- Mercado de massa (ex: processadores de texto, planilhas, aplicações financeiras)

Aplicações Web

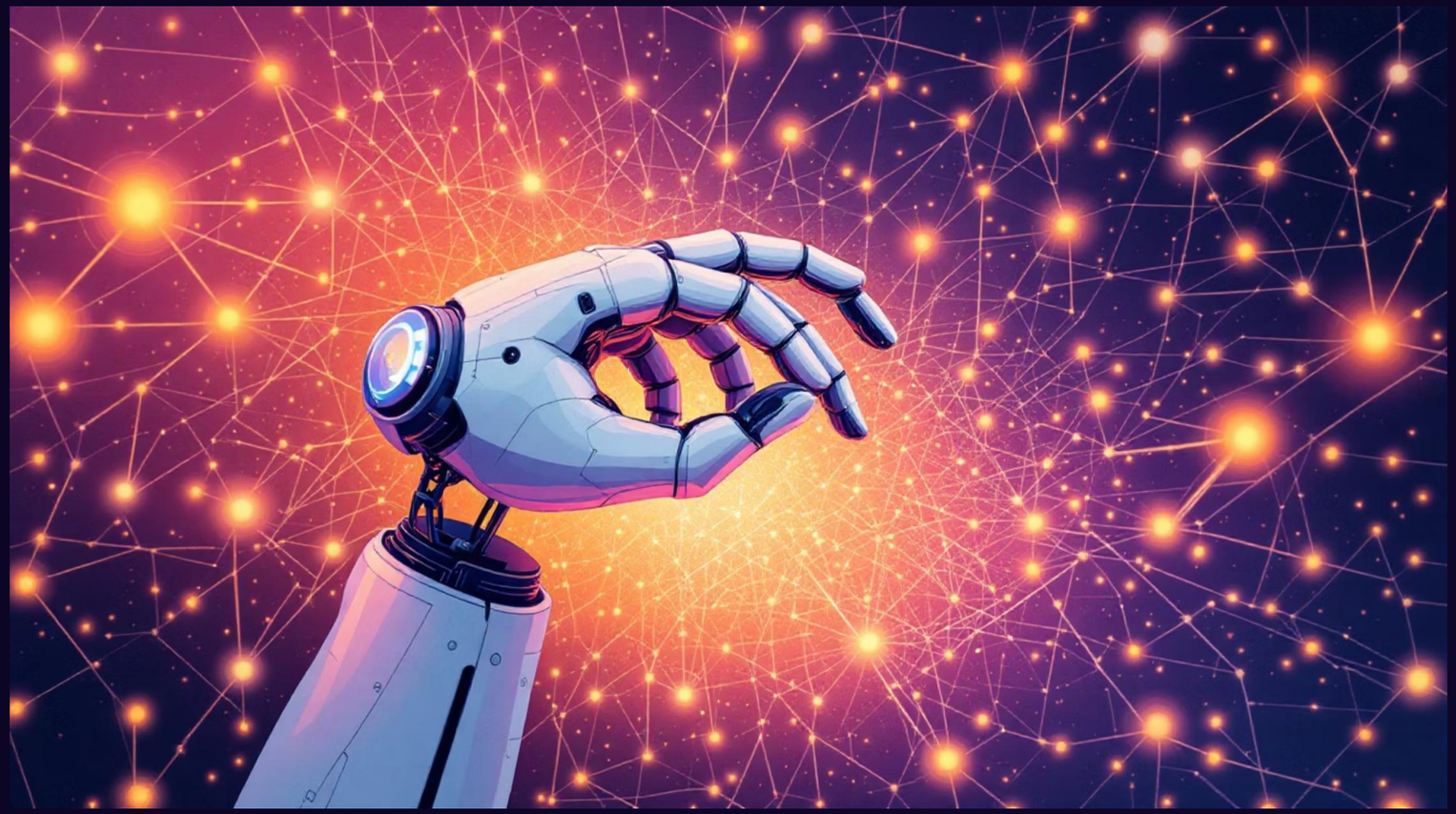
Funcionam na web, integradas com interface e banco de dados. Exemplos práticos do cotidiano:

- *Internet Banking*
- Declaração de Imposto de Renda



INTELIGÊNCIA ARTIFICIAL

Software para IA



Algoritmos não numéricos para problemas complexos

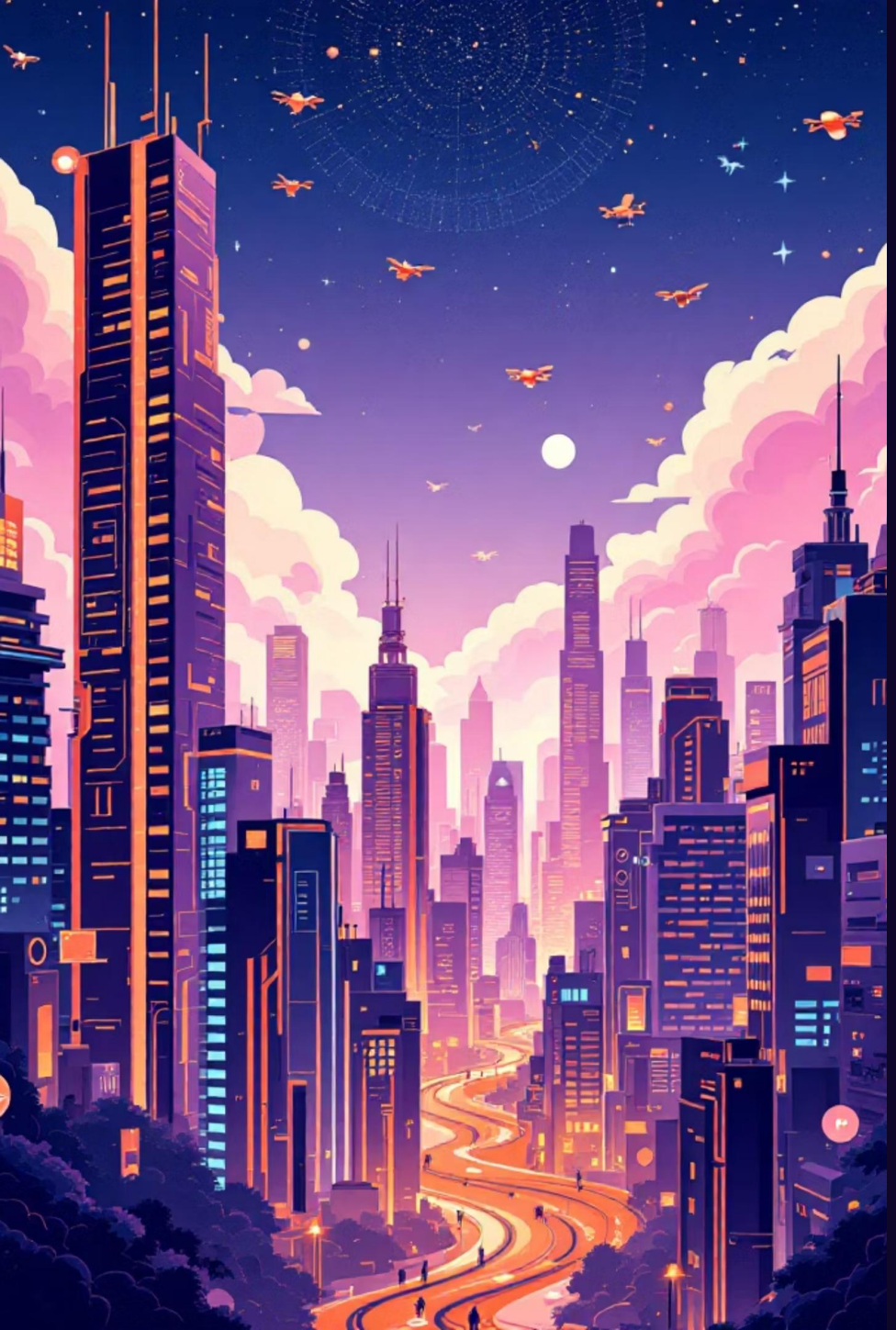
O *software* de inteligência artificial utiliza técnicas avançadas para resolver problemas que vão além do processamento numérico tradicional.

Robótica

Sistemas Especialistas

Reconhecimento de Padrões

Redes Neurais



Novas Aplicações de *Software*

Computação Ubíqua

Computação distribuída entre diferentes dispositivos (PCs, celulares, grandes sistemas) via redes *wireless*

Netsourcing

A Web como ferramenta de computação e fornecedora de conteúdo sob demanda

Software Livre

Distribuição de código fonte aberto para sistemas de aplicação colaborativos

Tendências Emergentes

Data mining, grid computing, cognitive machines e software para nanotecnologias

LEGADO

Software Legado

Sistemas antigos ainda em uso que precisam evoluir para acompanhar as demandas atuais. **Por que atualizar?**

1

Novo Ambiente

Adaptar o software para novas tecnologias computacionais

2

Novos Requisitos

Aperfeiçoar para implementar necessidades de negócio atualizadas

3

Interoperabilidade

Estender para integrar com sistemas e bancos de dados modernos

4

Rede

Rearquitetar para viabilizar uso em ambientes de rede



Mitos do *Software* — Gerência

Mitos de gerência envolvem **prazos, custos e qualidade**. Conheça os mais comuns:

Mito 1 — Manuais bastam

"Já temos um manual de padrões e procedimentos. Isso é suficiente." Na prática, manuais são ignorados ou estão desatualizados.

Mito 2 — Mais programadores

"Se estamos atrasados, adicionamos mais pessoas." Adicionar pessoas a um projeto atrasado o torna ainda mais atrasado (Lei de Brooks).

Mito 3 — Ferramentas de última geração

"Compramos os melhores computadores." Ferramentas sozinhas não garantem qualidade — processo e disciplina são essenciais.

Mito 4 — Terceirizar e relaxar

"Terceirizando posso relaxar." Sem acompanhamento constante, projetos terceirizados facilmente saem do controle.

COMUNICAÇÃO

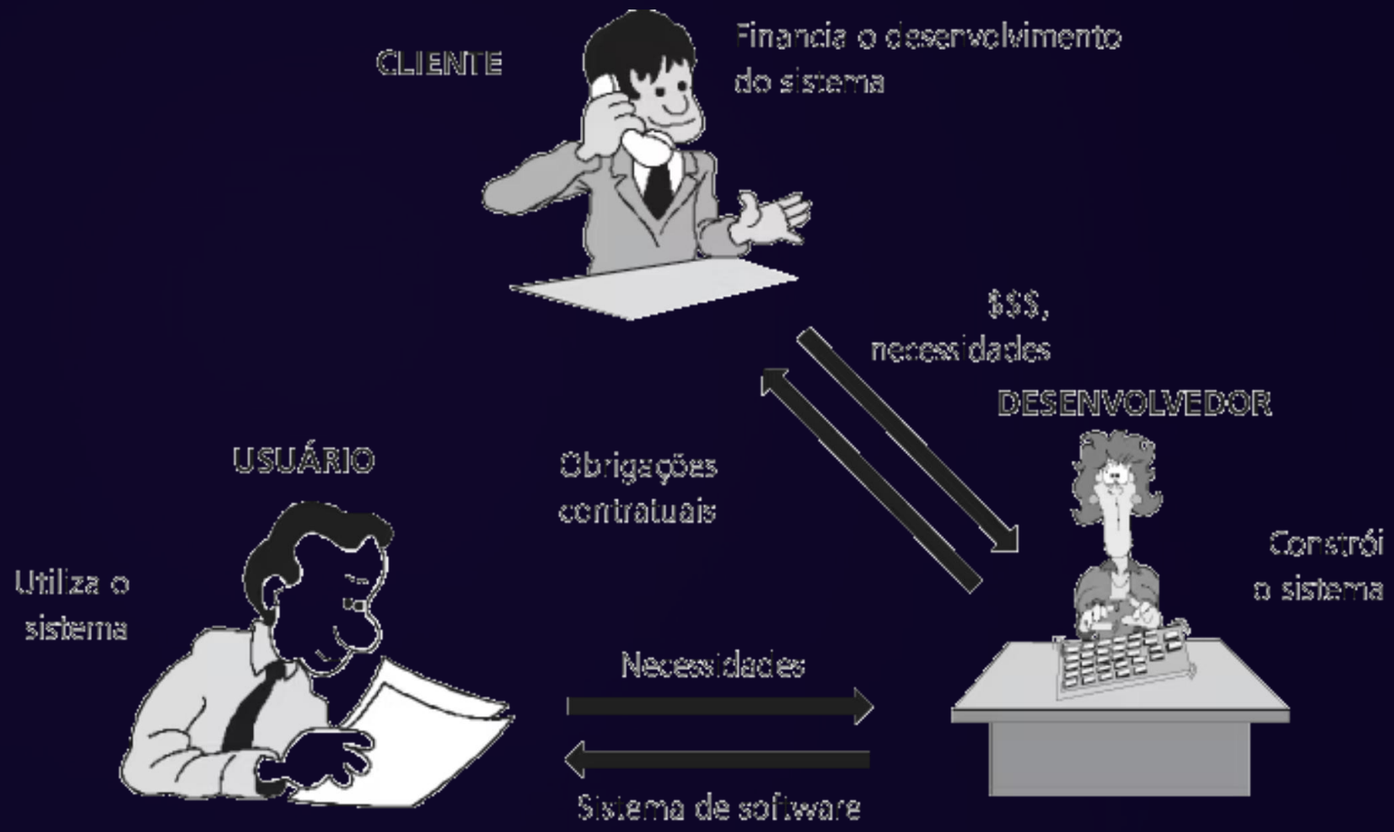
Mitos do *Software* — Cliente

Mito 5 — Objetivos gerais bastam

"Uma declaração geral dos objetivos é suficiente para se começar a escrever programas, podemos preencher os detalhes mais tarde."

Requisitos vagos levam a retrabalho, atrasos e custos elevados. Detalhar desde o início é fundamental para o sucesso do projeto.

Fonte: *Engenharia de Software: Teoria e Prática* — Shari Lawrence Pfleeger, 2004



COMUNICAÇÃO

Mitos do *Software* — Cliente (continuação)

> *"Os requisitos de projeto modificam-se continuamente, mas as mudanças podem ser facilmente acomodadas — o software é flexível."*

Realidade: O custo de mudanças cresce exponencialmente conforme o projeto avança. Alterações tardias podem custar até **100x mais** do que se fossem identificadas na fase de requisitos.



CULTURA DE PROGRAMAÇÃO

Mitos do *Software* — Profissional

1 Programa rodando = trabalho completo

Manutenção consome até 60% do esforço total. Entregar o programa é apenas o começo.

2 Qualidade só se avalia rodando

Revisões técnicas e inspeções de código identificam defeitos antes da execução.

3 Só o programa importa

Documentação, modelos e planos de teste são entregáveis essenciais de um projeto bem-sucedido.

4 Documentação atrasa o projeto

A engenharia de software produz documentação que **reduz** retrabalho e melhora a qualidade final.

Engenharia de Software

